Vocab Embedding 也要稀疏化吗?

目录

一、Over Encoding 1
二、Per-Layer Embeddings（PLE） 3
三、Engram 3
四、Qwen3.8 Flash Next 的消融 7

模型	核心做法	N-gram	地址映射	注入位置	Context-aware Gate	参数，比例
Over-Tokenized Transformer (OE)	**Hierarchical N-gram → Feature Hashing → sliced low-dim embeddings → projection → 输入 embedding**	1...N-gram	f(ngram) mod m + 多 sliced tables	模型输入	-	-
Gemma 4 E2B/E4B	**Per-Layer Embedding：每层独立 token embedding + context projection**	-	Token-ID lookup	每层	-	-
Engram-27B	**Tokenizer compression → N-gram → 多头 Hash → memory lookup → context-aware fusion → short causal Conv → 多 residual stream**	2/3-gram	compressed IDs + 8 hash heads/order	第 2、15 层	✓	5.7B，21.30%
LongCat-2.0	**Scale OE**	2–5 gram	Hash/mod + 4 splits/order	模型输入	-	135B，8.4%
DeepSeek-V4.1-Flash	**Scale Engram；删除 short Conv；Sinkhorn balancing**	2/3/4-gram	compressed IDs + 8 hash heads/order	第 2、15 层	✓	196B，26.2%
Qwen3.8-Flash-Next	**Scale Engram；去掉 tokenizer compression，直接 hash raw token IDs**	2/3-gram	raw token IDs + 8 hash heads/order	第 2 层	✓	51B，29.0%

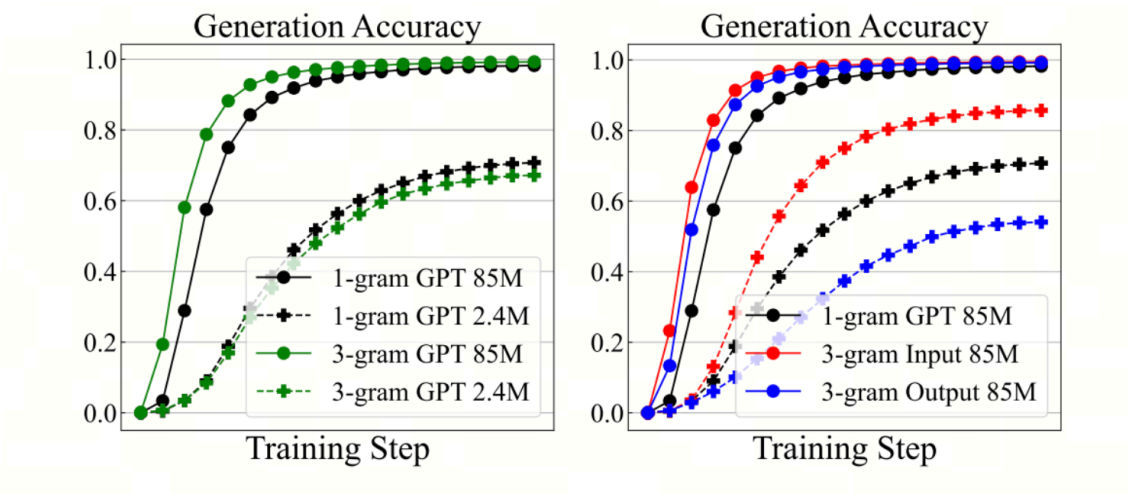
一、Over Encoding

Input Vocabulary 与 Output Vocabulary

在之前的 paper 中，就有发现：大 tokenizer 往往有利于大模型，却可能伤害小模型。

但是，tokenizer，或者说词表，其实有两个作用：**input vocabulary** 和 **output vocabulary**。

于是我们可以进一步解耦这个问题：到底是“大 input vocabulary”伤害小模型，还是“大 output vocabulary”伤害小模型？



Over Encoding（OE）发现，实际上是大 **output vocabulary** 导致的。

这里有一个很直觉的解释：input vocabulary 增大，本质是在增强上下文到 **representation 的映射能力**；而 output vocabulary 增大，则是在把预测问题变成更细粒度、更困难的分类任务。大模型有能力利用这种 supervision，小模型则容易 underfit。

从这个角度来讲，**input vocabulary 和 output vocabulary 不应该被绑定在一起 scaling**。

Hierarchical N-gram 与 Feature Hashing

做法上，论文保留原来的 BPE token 序列不变，但把每个位置的输入 embedding 从“只查当前 token”，改成**同时查当前 token 及其历史 n-gram**。

例如，当前位置为 c，前两个 token 是 a, b，则构造：

- **1-gram**：c；
- **2-gram**：(b, c)；
- **3-gram**：(a, b, c)。

先用下面的函数，把每个固定长度的 n-gram 唯一编码成一个巨大整数：

$$f(z_1, \dots, z_n) = \sum_{j=1}^{n} z_j V^{j-1},$$

其中 V 是基础词表大小。由于理论上有 V^n 种组合，不可能真的建 V^n 行 embedding，作者使用 **feature hashing**，直接计算 $f(\cdot) \bmod m$，只查一个 m 行的表 $E[f(\cdot) \bmod m]$，允许不同 n-gram 碰撞到同一行。

所谓 **hierarchical n-gram**，就是不是只用 3-gram，而是在同一个位置同时加入 $1+2+\cdots+n$-gram embedding。例如：

$$h_c = E_1[c] + E_2[f(b,c) \bmod m] + E_3[f(a,b,c) \bmod m]\,.$$

从而同时保留单 token、短局部组合和更长局部组合的信息。

Sliced Low-Dimensional Embeddings

进一步，对于每一种 n-gram，作者不是只建一个宽的 $m \times d$ 表，而是使用 **sliced low-dimensional embeddings**：把同样的参数预算拆成 k 个较窄的 $m_i \times (d/k)$ 表，每次 lookup 出低维向量后，用 $W_i \in \mathbb{R}^{(d/k)\times d_{\text{model}}}$ 投影回模型维度，再相加：

$$E_{\text{slice}}(x) = \sum_{i=1}^{k} E_i[x \bmod m_i] W_i.$$

关键是这些 slice 的 m_i 故意设得略有不同，如 $m, m+2, m+4$。于是，同一个 n-gram 会得到 $x \bmod m$、$x \bmod (m+2)$, ... 多组不同的 hash 地址：即使它在一个表里和别的 n-gram 撞车，也很难在所有表里同时撞车。

最终就是 **BPE embedding + 多层级 n-gram × 多个不同 hash view 的低维 embedding**，全部相加后送进原封不动的 Transformer。因此，增加的是大量稀疏 lookup 参数，而几乎不增加 Transformer 主干 FLOPs。

从上述流程也可以看出，OE 的 **hash 地址和稀疏查表只依赖 token ID**，不需要等待 contextual hidden state，特别适合放到 CPU，甚至提前计算。查表后的 Linear 投影仍有矩阵运算，但 OE 参数虽然很大，每个 token 只访问极少几行。换个视角来说，这也是一种对 input embedding 的稀疏化。

```
2-gram (b,c)   -> hash_i -> Emb_i -> Linear_i -> sum_i --+
3-gram (a,b,c) -> hash_i -> Emb_i -> Linear_i -> sum_i --+-> SUM
1-gram c      -> Token Embedding ------------------------+     |
                                                               v
                                                  Transformer -> LM Head
```

每条 n-gram 分支里的 $i = 1, \ldots, k$ 对应多个 slice；先在分支内求和，再与基础 token embedding 相加。

二、Per-Layer Embeddings（PLE）

Gemma 3n、Gemma 4 E2B / E4B 都使用了 **Per-Layer Embeddings**。

Motivation 为：**减少 backbone 做"静态记忆"的负担，用 memory scaling 换 compute scaling**，不同深度可以学不同的 token prior。前者也是后续 Engram 的 motivation。

做法上，除了普通 token embedding 之外，再给每个 token 在每一层都准备一份独立的低维 embedding，例如：

$$\text{token} \times \text{layer} \longrightarrow \text{256-d vector}.$$

模型运行到第 m 层时，会取出这个 token 对应的第 m 层 PLE，用当前 contextual hidden state 乘 gate 矩阵，再过 GELU 产生一个 gate，选择性读取其中的信息，然后投影回 hidden dimension，加到 residual stream。

它的本质可以理解为一种 **layer-specific token memory**：让深层网络不必完全依赖前面几十层去保存原始 token 信息，而是每层都能直接访问一份针对该层专门学习的静态记忆。

这样可以用大量、稀疏访问的 embedding 参数提升模型容量，同时只增加较少的实际计算。

三、Engram

Motivation：Conditional Memory

Engram 比较难得，是一篇在技术上和写作上都很棒的 paper，并且在吸取了 OE 与 PLE 的设计经验后进一步创新。我们来细读一下。

首先是 motivation 上，将大模型的计算分为：

1. **动态、组合性的计算**：逻辑推理、数学推导、代码、长距离关系等，需要真正的深层 block 计算。
2. **静态、局部、重复的模式**：人名、地名、固定短语、习语、常见局部组合等，本质上更类似"看到 key，取出 value"，并不需要很深的计算来获取这类 factual knowledge。

在此之外，作者还有另外更大的野心：希望找到第二个 sparsity 轴，即 Conditional Memory；以及在 dense compute 和 MoE experts 之外，找到 LLM 的第三种容量 scaling 方式：增加静态、稀疏访问的 memory。

从 N-gram Lookup 到 Residual Injection

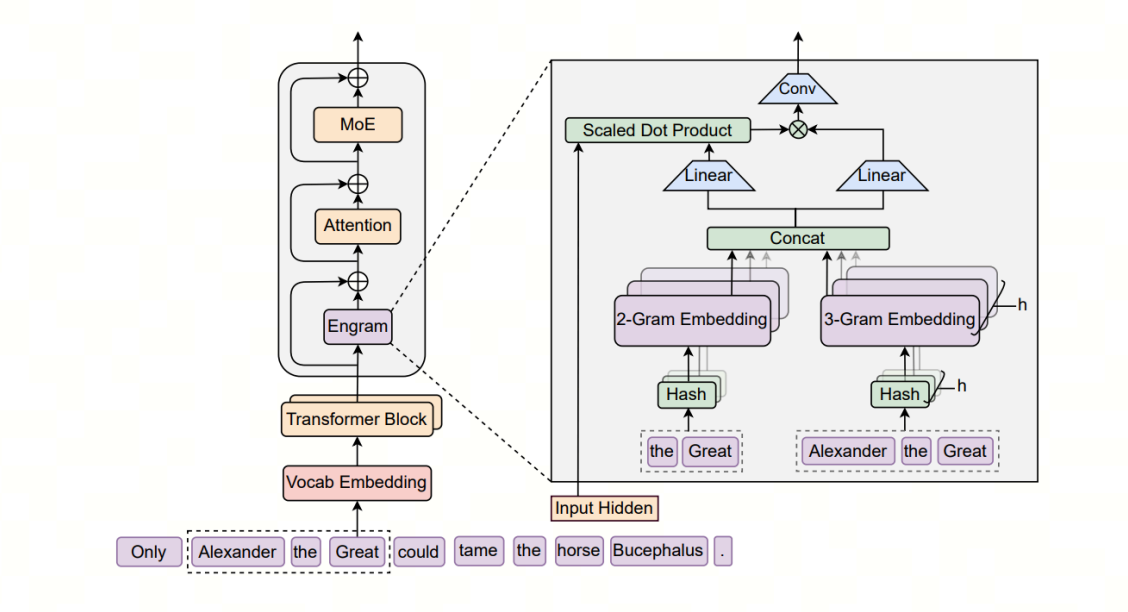


Figure 1 | **The Engram Architecture.** The module augments the backbone by retrieving static *N*-gram memory and fusing it with dynamic hidden states via context-aware gating. This module is applied only to specific layers to decouple memory from compute, leaving the standard input embedding and un-embedding module intact.

因为大模型 tokenizer 中，许多文本形式近似等价的 **token 却拥有完全不同的 ID**，例如大小写差异、是否带前导空格等，Engram 首先对 token 做 **canonicalization**，将这些变体映射到更统一的表示空间。

随后，在当前位置构造 2-gram、3-gram 等局部短语，并通过多个独立 hash 函数映射到超大的 embedding table 中进行查表：

$$e_t = \mathrm{Concat}\Big(\big\{E_{n,k}[\phi_{n,k}(g_{t,n})]\big\}_{n,k}\Big).$$

查出的 memory e_t 经过共享的 Value projection W_V，得到：

$$v_t = W_V e_t.$$

在 mHC 中，同一层的多个 residual stream 共享这份 memory 和 W_V，但每个 branch m 拥有独立的 $W_K^{(m)}$。模型利用当前 branch 的 hidden state $h_t^{(m)}$ 与 memory key 计算 **context-aware gate**：

$$\alpha_t^{(m)} = \sigma\left(\frac{\mathrm{RMSNorm}(h_t^{(m)})^\top \mathrm{RMSNorm}(W_K^{(m)} e_t)}{\sqrt{d}}\right).$$

从而得到该 branch 实际读取的 memory：

$$\tilde{v}_t^{(m)} = \alpha_t^{(m)} v_t.$$

最后，原文中声称是为了进一步扩大局部感受野并增强非线性，在 gated memory 序列 $\tilde{V}$ 上加入一个轻量的 **short depthwise causal convolution**：

$$Y = \mathrm{SiLU}\Big(\mathrm{Conv1D}\Big(\mathrm{RMSNorm}(\tilde{V})\Big)\Big) + \tilde{V}.$$

（这玩意儿我很存疑。）

最终 Y 再写回对应的 residual stream。整体上就是：**canonicalization → N-gram hash lookup → context-aware gated retrieval → local causal convolution refinement → residual injection**。

Infra：Host Memory 与 Prefetch

与 OE 同理，由于 hash 地址只依赖 token ID，memory 可以提前在 CPU / Host DRAM 查表并异步搬到 GPU，与前层计算重叠。因此，可以把参数规模扩得很大，而每个 token 的 lookup 数量保持不变；投影、gate 与卷积仍有少量计算开销。

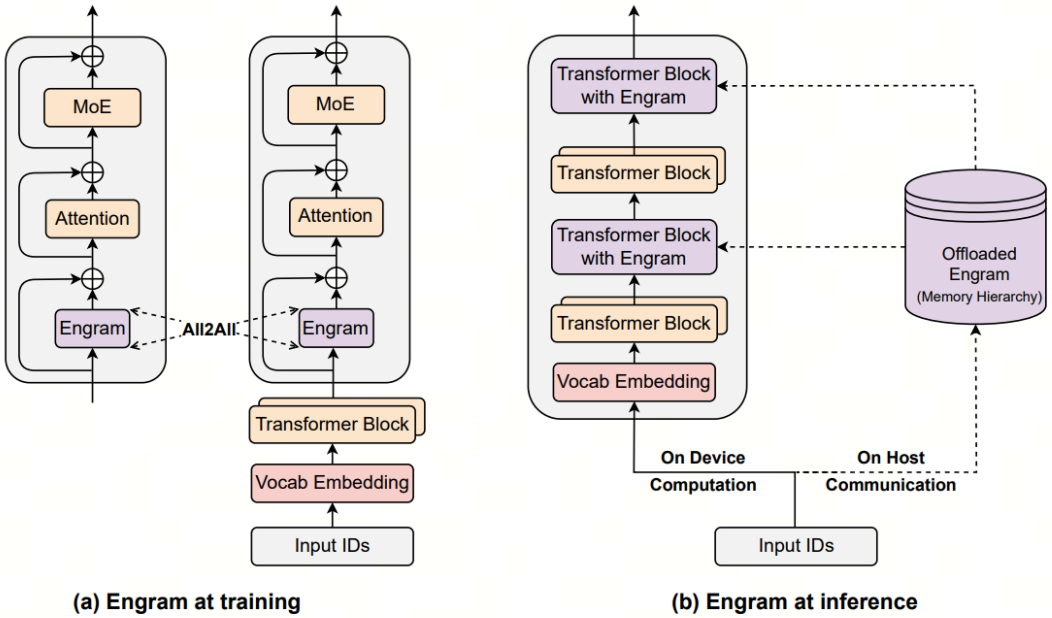


Figure 2 | **System implementation of Engram.** (a) Training Phase: The massive embedding tables are sharded across available GPUs. An All-to-All communication primitive is employed to retrieve active embedding rows across devices. (b) Inference Phase: Engram tables are offloaded to host memory. By exploiting the deterministic retrieval logic, the host asynchronously prefetches and transfers embeddings, overlapping communication with the on-device computation of preceding Transformer blocks.

有趣的实验问题与结论

实验结果就无需多言。接着，我们看看文章里有哪些有趣的实验问题与结论。

1. **固定预算下，MoE 和 Engram 的参数怎么分?**

   固定总参数和激活参数，只改变 sparse budget 在 MoE 与 Engram 之间的分配。结果呈明显 U 型：纯 MoE 不是最优，大约 **75%-80% 给 MoE、20%-25% 给 Engram** 效果最好。

2. **Engram 可以 scale 吗?**

   可以。固定 backbone，只不断扩大 Engram table，validation loss 持续下降，并呈较稳定的 scaling 趋势。更 nice 的是：memory table 可以变得很大，但每个 token 只查固定数量的 row，因此参数规模增长并不会同比增加 FLOPs。所以，memory size 可以成为独立于模型计算量之外的新 scaling axis。

3. **N-gram memory 能改善 reasoning 吗?**

   可以！虽然 Engram 本身不是在"做推理"，但有了它，也许前几层不必花计算重新组合静态实体和短语，因此能更早进入高层语义处理。LogitLens 和 CKA 显示，Engram 模型在更浅层就能形成接近 baseline 更深层的表示，因此作者称其提升了 **effective depth**：把更多真正的 Transformer 深度留给组合推理、数学和代码。

4. **Engram 能改善长上下文吗?**

   可以！普通 Attention 不仅要处理长程依赖，还要承担大量局部 pattern 建模；Engram 把一部分 local dependency 直接交给 memory lookup。这样，Attention capacity 可以更多用于 global dependency。（写到这里，不禁再次让我想到 Zeyuan 的语言模型物理学。）

5. **Engram 应该放在哪一层？哪些组件真正有用?**

   这也是一个需要 trade-off 的事情。直觉来想，太早时 hidden state 缺少 context，可能 gate 不准；太晚又已经浪费了前层计算。文章实验发现，在第二层放置最好，把预算拆成多个 Engram、分布在不同深度，还能进一步提升。

   消融上，**context-aware gate、tokenizer compression、mHC branch-specific fusion** 最关键；Short Causal Conv 只有小幅增益（于是 DS4.1 干脆删了）。

6. **Engram 到底学了什么?**

   关闭 Engram 后，事实知识类任务性能大幅下降，而阅读理解保留得更多。这说明模型自然形成了功能分工：Engram 更偏向存储实体、事实、固定短语等静态 parametric knowledge；Transformer backbone 更偏向上下文处理、组合与推理。

当然，以上结论都是在小 size 模型上做的。在现在的发版模型中，具体 Engram 的配置也有了一些变化。

四、Qwen3.8 Flash Next 的消融

在这里，我简单提下 Qwen3.8 Flash Next 中的消融结论。

Table 8: Effect of N-gram vocabulary scaling under a fixed total model parameter budget. Vocabulary scales are measured relative to the base tokenizer vocabulary size (250K). The number of MoE experts is adjusted to offset the additional N-gram embedding parameters.

Vocab. Scale	Loss	Uncheatable	Knowledge			STEM		Reasoning	Chinese		Multilingual
(Param. Ratio)		PPL	MMLU	MMLU-Pro	SuperGPQA	MATH	GSM8K	BBH	C-Eval	CMMLU	MMMLU
None (0%)	1.202	5.55	**68.25**	44.38	25.22	46.02	**74.79**	**65.71**	70.78	73.01	**58.32**
5× (10%)	1.200	**5.54**	68.15	44.49	24.25	45.64	72.86	65.39	70.93	**73.44**	57.49
10× (25%)	**1.197**	5.55	67.71	**44.66**	**25.64**	**46.56**	73.65	65.54	70.71	73.31	56.22
30× (50%)	1.201	5.59	67.75	42.61	24.18	44.62	74.45	65.55	**72.49**	73.28	56.66

1. **N-gram 层数放置**。浅层总体较强，但中层、深层也有竞争力；把固定参数预算拆成两层并没有稳定收益。因此，最终选择只放 **Layer 2**。放在 Layer 2，可以在计算 Layer 1 时，从 host memory 提前 prefetch N-gram embedding。
2. **固定总参数预算**。约 25% 参数给 N-gram 时，training loss 最好，但 downstream 评估上并没有对应的 25% 最优点。
3. **固定 backbone，额外增加 N-gram 参数**。从 loss 上来看，N-gram 可以持续 scale。但是，"**loss 越低 ≠ downstream 越好**"：reasoning benchmark 到一定规模之后会持平，甚至下降。
4. **中文 benchmark**。 N-gram 对中文 benchmark（如 C-Eval 和 CMMLU）的 scaling 特别稳定。

Table 9: Effect of N-gram vocabulary scaling. Vocabulary scales are measured relative to the base tokenizer vocabulary size (250K). Larger vocabularies increase the total number of parameters.

		Knowledge			STEM		Reasoning	Chinese		Multilingual
Vocab. Scale	Loss	MMLU	MMLU-Pro	SuperGPQA	MATH	GSM8K	BBH	C-Eval	CMMLU	MMMLU
None	1.585	62.78	33.43	20.97	32.52	59.21	53.40	66.91	68.10	54.06
20×	1.553	64.14	34.46	21.83	**37.38**	**65.09**	57.13	71.75	72.29	55.94
50×	1.541	64.71	35.80	21.49	37.32	64.00	**57.56**	72.12	72.48	56.64
100×	1.534	64.70	**35.87**	**21.93**	36.98	63.08	56.03	73.75	72.73	**56.65**
200×	**1.526**	**64.85**	35.21	21.11	35.34	62.96	56.23	**74.94**	**73.24**	55.82

此外，LongCat-2.0 的选择也略有不同，在这里我不再赘述，感兴趣的可以去看他们的技术报告。